



Lab_4_VibeCoding...
PDF



VibeCodingGitRep...
PDF



README.md
Document

I am working on my Software Engineering Lab 4 assignment on Vibe Coding.

I will first upload the official lab documents and the assigned repository/README.

Please read all the uploaded materials carefully and use them as the primary source for this entire conversation.

Do not start modifying or suggesting code yet.

First, identify and summarize:

1. The objective of the lab
2. The exact tasks I am required to complete
3. The required deliverables
4. The rules/restrictions given by the lab
5. The features that the Q*bert README specifically asks me to implement
6. What evidence/screenshots/videos/chat history I need for submission

Do not add requirements that are not present in the uploaded materials.

After reviewing everything, wait for my next instruction.

I reviewed all three uploaded materials carefully and will use them as the **primary source for this conversation**. I have not started modifying or suggesting code.

1. Objective of Lab 4

The official lab handout defines the objective as:

- Use **Vibe Coding tools** to fix broken code and add features.
- Learn how to prompt effectively enough to get the job done in **under three to four attempts**.
- Clone the assigned repository, make the required changes, and push the result to your **personal repository**.

- The assignment is explicitly **individual work**.  Lab_4_VibeCoding_handout

The lab duration specified is **60 minutes**.  Lab_4_VibeCoding_handout

The Q*bert README additionally frames the activity as using an **LLM such as ChatGPT or Claude as a debugging and pair-programming partner**, with iterative LLM suggestions followed by your own critical code review.  README  README

2. Exact tasks you are required to complete

Task 1: Fix the pyramid projection bug

The README says the pyramid is visibly skewed from the first frame instead of forming a symmetric triangle.

You are specifically directed to investigate the arithmetic in:

```
cube_center
```

and determine whether the row offset should use **integer division or true division**.  README

Task 2: Implement `cube_palette(level)`

This function is currently empty.

The README specifies that it:

- Is called once per frame from `draw`
- Receives the current level
- Should return a list of **TARGET + 1 colors**, currently 3
- Each color is an `(r, g, b)` tuple
- The colors represent the stages from unpainted to fully painted
- It may alternatively return `None`, which keeps `DEFAULT_PALETTE`
- The suggested idea is to have a different three-color palette for each level.  README

Task 3: Implement `on_cube_completed(cell)`

This function is also currently empty.

It is called from `paint()` when a particular cube **first reaches its target stage**.

Important detail: it is **not** called on every hop onto the cube, only when that hop actually completes the cube.

The supplied idea is either:

- a brief flash on the completed cube, or
- a small bonus beyond the existing **25 points per paint step**.

Its return value is ignored.  [README](#)

Task 4: Implement `bonus_life_threshold()`

This function is called every frame from `update()` .

It:

- Takes no arguments
- Returns an integer score threshold, or `None` to disable bonus lives
- Awards one life when the score crosses a multiple of that threshold for the first time
- Already has its bookkeeping implemented through `self.bonus_awarded`

The README gives **1000** as the suggested threshold.  [README](#)

3. Required deliverables

There are two relevant source documents here.

Official Lab Handout

The handout says that under your Lab-4 folder in your **Lab repo**, you must upload:

1. **Before video**
2. **After video**
3. **Updated code**
4. **Exported Chat history as a DOC/PDF**  [Lab_4_VibeCoding_handout](#)

Q*bert README

The repository's own submission checklist specifies the following three things:

1. **10-second gameplay video before changes**, showing the bug/broken behavior
2. **10-second gameplay video after changes**, showing the bug fixed and new features working
3. **Chat/LLM used page link**, containing the complete chat history  [README](#)

So I will keep both requirements in mind rather than silently dropping either one.

4. Rules and restrictions

From the official lab handout:

- **Individual assignment**
- Work individually.  [Lab_4_VibeCoding_handout](#)
- You must use a **Vibe Coding tool**, with examples given as ChatGPT, Gemini, Claude, etc.
 [Lab_4_VibeCoding_handout](#)
- Go to the assigned repository.
- Read its README to understand the deliverables.
- Clone or fork the repository.
- Run the Python-based game/puzzle.
- Record the **10-second video before making changes**.
- Write prompts and fix the broken code.
- Write prompts to add the README-listed features **one by one**.
- Record another **10-second video after all changes**.
- Push the completed code to your repository.
- **Do not raise a PR to the main SETAPESU26 repository.**  [Lab_4_VibeCoding_handout](#)

The Q*bert README additionally requires the tasks to be completed through an **iterative LLM suggestion + critical code review process**.  [README](#)

Repository setup specified by the README

The supplied Q*bert project requires:

- Python 3.10+
- pygame
- Run using `python game.py`
- Controls:
 - Left
 - Up
 - Down
 - Right
 - R to reset
 - Space to continue after clearing a level  [README](#)

I am treating these as repository-specific setup information, not additional lab requirements.

5. Features specifically required by the Q*bert README

The README describes the starting game as already having:

- A pyramid of cubes
- Diagonal hopping
- Cube painting toward a target color
- Coily chasing the player
- Red balls rolling downward
- Life loss from falling off the pyramid or bumping an enemy while grounded
- Levels, lives and scoring
- Winning when every cube reaches its target color  README

The changes specifically left for you are:

Task	Required change
1	Fix the skewed pyramid projection
2	Implement <code>cube_palette(level)</code>
3	Implement <code>on_cube_completed(cell)</code>
4	Implement <code>bonus_life_threshold()</code>

The README calls the project one with **one deliberate bug and three optional features left as empty functions**, while the lab instructions explicitly tell you to add **all features listed in the README one by one**.  README  Lab_4_VibeCoding_handout

6. Expected behavior after your work

The Q*bert README specifies that the finished game should exhibit:

- A **symmetric triangular pyramid**
- Landing on a cube advances its color by **exactly one stage per hop**
- Falling off the edge makes the player fall and costs a life
- Coily chases **toward** the player's current position
- Painting every cube to its target color wins the level
- Running out of lives ends the game  README

These are the behaviors we should use later when checking whether the implementation actually works.

7. Evidence / screenshots / videos / chat history

The sources explicitly require **videos** and **chat history**, but they do **not** explicitly require a separate collection of screenshots.

Before changes

You need a **10-second gameplay recording before making changes**, specifically showing the broken behavior.  [Lab_4_VibeCoding_handout](#)  [README](#)

For Q*bert, that means the recording should visibly capture the initial broken/skewed pyramid behavior.

After changes

You need another **10-second gameplay recording after all changes**, showing:

- The projection bug fixed
- The new features working  [Lab_4_VibeCoding_handout](#)  [README](#)

Chat/LLM evidence

The lab handout asks for:

Chat history exported as a doc/pdf

while the Q*bert README specifically asks for:

Chat/LLM used page link, with the complete chat history.  [Lab_4_VibeCoding_handout](#)  [README](#)

I will therefore preserve the **actual iterative prompting process** in this conversation rather than jumping straight to a finished solution.

Code

The updated Q*bert code must also be submitted in the Lab-4 folder according to the official handout.

 [Lab_4_VibeCoding_handout](#)

Your assigned repository

The repository mapping document assigns:

Q*bert

https://github.com/SETAPESU26/35_qbert

to **G V N SREEHARSHA, USN PES1UG24CS163.**  [VibeCodingGitRepoLinks-Sec-C](#)

So the materials establish that **35_qbert** is your assigned repository.

One important boundary for our next steps

At this stage, I have **not** proposed fixes, written code, changed anything, or invented additional submission requirements.

I have the lab requirements, your assigned *Qbert repository information*, and the Qbert README requirements mapped out. **I'll wait for your next instruction.**

Now I want to work on the assigned Q*bert repository.

I will provide the current repository/code next.

First inspect the existing code and identify:

1. How the game currently works
2. Which part contains the deliberate bug
3. Where each required feature should be implemented
4. Any existing functions/placeholders intended for the required features

Do not modify the code yet.

Explain what needs to be changed and why, based primarily on the README and lab instructions.

👍 Got it. I'll inspect the **current Q*bert code only** and won't modify it yet.

Once you upload/provide the repository files, I'll trace:

1. Current game flow

- Initialization
- Game loop
- Player movement/hopping
- Pyramid/cube rendering
- Cube painting and scoring
- Coily/enemy behavior
- Lives, levels, win/game-over logic

2. Deliberate bug

- Locate the relevant code, especially the `cube_center` arithmetic mentioned in the README.
- Explain exactly why the current implementation produces the skewed pyramid.

3. Required feature locations

- `cube_palette(level)`
- `on_cube_completed(cell)`
- `bonus_life_threshold()`
- How each is called by the existing game.

4. Existing placeholders

- Identify the exact empty/stub functions already provided for the assignment.
- Explain their inputs, expected outputs, and when they are invoked.

5. Change map

- I'll give you a clear "**current code** → **required change** → **reason**" map.
- I will distinguish what is explicitly required by the README from anything that is merely an implementation possibility.

I won't write replacement code, modify files, or start implementing anything until you explicitly tell me to proceed.

Upload the current Q*bert repository/code next.

 [https://github.com/SETAPESU26/35_qbert?](https://github.com/SETAPESU26/35_qbert?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://github.com/SETAPESU26/35_qbert?utm_source=chatgpt.com)

I inspected the **current main branch of SETAPESU26/35_qbert** , including its `README.md` and `game.py` . The repository currently contains only `.gitignore` , `README.md` , and `game.py` , with `game.py` being the single-file implementation described by the README.  GitHub

I have not modified anything.

1. How the current game works

The game is a small Pygame, object-oriented Q*bert clone. The main flow is:

 Plain text

```
main()
↓
Game()
↓
Game.reset()
↓
Game.respawn()
```

```

↓
Main game loop
├─ keyboard input → hop()
├─ update(dt)
│   ├── update player hop
│   ├── paint landed cube
│   ├── check bonus-life threshold
│   ├── spawn/move Coily and balls
│   ├── detect collisions
│   ├── detect falling
│   └─ detect level completion
└─ draw(screen)

```

The README confirms that the game consists of a pyramid of cubes, diagonal hopping, cube painting, Coily, red balls, lives, levels and scoring. [GitHub](#)

Pyramid representation

The pyramid is represented using:

</> Python

```

ROWS = 7
TARGET = 2
CELLS = {(r, c) for r in range(ROWS) for c in range(r + 1)}

```

So the rows contain:

</> Plain text

```

Row 0:      1 cube
Row 1:      2 cubes
Row 2:      3 cubes
Row 3:      4 cubes
...
Row 6:      7 cubes

```

Each cube has a (row, col) coordinate.

The cube's position on the screen is calculated by `cube_center(row, col)`.

2. Player movement and hopping

The four controls are mapped to diagonal movements:

</> Python

```
KEY_HOPS = {
    pygame.K_LEFT: (-1, -1),
    pygame.K_UP: (-1, 0),
    pygame.K_DOWN: (1, 0),
    pygame.K_RIGHT: (1, 1)
}
```

When a key is pressed, `Game.hop()` calculates the candidate cell and starts a hop.

The important point is that the player can even start a hop toward a position outside the pyramid.

`Hopper.update()` eventually detects that the resulting cell is not in `CELLS` and changes the player into a falling state.

That corresponds directly to the README's expected behavior that hopping off the pyramid causes the player to fall and lose a life. [GitHub](#)

3. Cube painting and scoring

The central function here is:

</> Python

```
Game.paint(cell)
```

Its current logic is:

</> Plain text

```
landing on cube
  ↓
is cube valid and not already complete?
  ↓
increase stage by 1
  ↓
+25 score
  ↓
if stage reached TARGET
```



```
on_cube_completed(cell)
```

The existing code already prevents a cube from progressing beyond `TARGET` .

This matches the README's expected behavior that landing on a cube advances its color **one stage per hop**.  [GitHub](#)

The cube stages are:

 **Plain text**

```
stage 0 → stage 1 → stage 2
```

because:

 **Python**

```
TARGET = 2
```

and the default palette contains three colors:

 **Python**

```
DEFAULT_PALETTE = [  
    (90, 160, 220),  
    (190, 120, 70),  
    (100, 210, 140)  
]
```

4. The deliberate bug

The README specifically tells us to inspect `cube_center()` and the arithmetic involving `row` and `col` .



The current implementation is:

 **Python**

```
def cube_center(row, col):  
    return pygame.Vector2(  
        WIDTH / 2 + (col - row // 2) * CUBE_W,  
        90 + row * CUBE_H  
    )
```

This is the deliberate bug.

The problematic part is:

</> Python

```
row // 2
```

The README explicitly hints that we need to determine whether this should use **integer division** or **true division**. [GitHub](#)

For an isometric pyramid, the row's horizontal offset needs to move by **half a cube width per row**.

With integer division:

</> Plain text

```
row // 2
```

the offset behaves like:

</> Plain text

```
row 0 → 0  
row 1 → 0  
row 2 → 1  
row 3 → 1  
row 4 → 2  
...
```

That means odd-numbered rows don't receive the required half-width shift.

For example, row 1 contains two cubes:

</> Plain text

correct conceptual positions:

[-36] [+36]

but integer division makes the row offset behave as though the center adjustment were zero:

</> Plain text

[0] [72]

So the two cubes aren't centered around the pyramid's vertical axis.

Therefore

The deliberate bug is specifically located in:

</> Plain text

```
cube_center()
  ↓
(col - row // 2)
  ↑
problematic
```

The README's description of the rows drifting sideways and breaking triangular symmetry corresponds directly to this expression. [GitHub](#)

We should not change it yet. We have identified the exact location and reason.

5. Required Feature 1: `cube_palette(level)`

There is already an explicit placeholder:

</> Python

```
def cube_palette(level):
    """Return a list of TARGET + 1 (r, g, b) colours for the cube stages, or None for the default."""
    pass
```

So this is **not a new function we need to invent**. The repository deliberately provides it for the assignment.

It is used in:

</> Python

```
def draw(self, screen):
    ...
    colors = cube_palette(self.level) or DEFAULT_PALETTE
```

Therefore the flow is:

</> Plain text

```
draw()
↓
cube_palette(self.level)
↓
returns custom palette
    OR
returns None
↓
draw_cube()
↓
colors[self.stages[cell]]
```

The README requires this function to return three RGB colors, one for each cube stage, or `None` to retain the default palette. It specifically suggests using different palettes for different levels.  [GitHub](#)

Where we need to work later

Function:

</> Plain text

```
cube_palette(level)
```

Location: top-level function in `game.py` .

No changes are currently needed elsewhere to connect it, because the connection already exists.

6. Required Feature 2: `on_cube_completed(cell)`

Again, the repository already contains the intended placeholder:

</> Python

```
def on_cube_completed(cell):  
    """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""  
    pass
```

And `Game.paint()` already calls it:

</> Python

```
if self.stages[cell] == TARGET:  
    on_cube_completed(cell)
```

This is particularly important because the existing code already ensures the function is called **only when the cube reaches the target stage**.

The README explicitly says:

not on every hop onto it, only the hop that finishes it.  [GitHub](#)

So we shouldn't redesign the painting system.

Where we need to work later

</> Plain text

```
paint(cell)  
  ↓  
cube reaches TARGET  
  ↓  
on_cube_completed(cell)
```

The placeholder itself is where the feature belongs.

The README gives possible directions such as:

- brief flash
- small bonus beyond the existing 25 points

These are examples provided by the assignment, not additional mandatory requirements.  [GitHub](#)

7. Required Feature 3: `bonus_life_threshold()`

There is already a placeholder:

</> Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""  
    pass
```

More importantly, the rest of the implementation for the feature is **already present**.

Inside `update()` :

</> Python

```
threshold = bonus_life_threshold()  
  
if threshold and self.score // threshold > self.bonus_awarded:  
    self.bonus_awarded = self.score // threshold  
    self.lives += 1
```

And `self.bonus_awarded` is initialized in `reset()` :

</> Python

```
self.bonus_awarded = 0
```

So the architecture is already:

</> Plain text

```
score changes  
  ↓  
update()  
  ↓  
bonus_life_threshold()  
  ↓  
threshold  
  ↓  
check score / threshold  
  ↓  
increase lives
```

The README explicitly says that the bookkeeping is already implemented and that we only need to choose the threshold. Its suggested value is:

</> Plain text

```
1000
``` :chatgpt-content-reference{index="10"}
```

So this feature does **\*\*not\*\*** require us to build an entire bonus-life system.

---

# 8. Coily behavior is already implemented

The README mentions Coily's chase behavior as part of the existing game and expected behavior.

The relevant code is:

```
```python
goal = cube_center(*self.player.cell)

self.coily.start(
    min(
        options,
        key=lambda n: cube_center(*n).distance_squared_to(goal)
    )
)
```

This means Coily considers valid neighboring cells and selects the one whose screen position is closest to the player's current position.

So the current code already contains the intended **toward-the-player** logic.

The assignment's four explicit tasks do **not** ask us to rewrite Coily.

The README's expected behavior confirms that Coily should chase **toward** the player's current position rather than away from it.  [GitHub](#)

9. Level completion

After updating the player, enemies, collisions, etc., `Game.update()` checks:

</> Python

```
if all(stage >= TARGET for stage in self.stages.values()):  
    self.score += 500  
    self.state = "win"
```

So completing every cube changes the state to "win" .

The rendering code then displays:

</> Plain text

LEVEL CLEAR! Press Space

When Space is pressed during the win state:

</> Python

```
game.level += 1  
game.reset(full=False)
```

So the existing game already has the basic level progression required by the README.

10. Game-over/lives system

The game starts with:

</> Python

```
self.level, self.score, self.lives = 1, 0, 3
```

When the player loses a life:

</> Python

```
self.lives -= 1
```

If lives reach zero:

</> Python

Otherwise, the player is respawned.

So the life system is already implemented. The only new life-related task is the **bonus-life threshold function**.

11. Exact change map

Here is the cleanest map of the current repository:

Assignment requirement	Existing location	Current status	What eventually needs changing
Fix pyramid projection	cube_center()	Deliberate bug	Correct the row division used in horizontal offset
Level-dependent colors	cube_palette(level)	pass placeholder	Implement the required palette behavior
Cube completion effect	on_cube_completed(cube)	pass placeholder	Implement a completion effect/bonus
Bonus life	bonus_life_threshold()	pass placeholder	Return the chosen score threshold
Cube painting	Game.paint()	Already implemented	No structural change required
Palette integration	Game.draw()	Already connected	No structural change required
Completion callback	Game.paint()	Already connected	No structural change required
Bonus-life bookkeeping	Game.update()	Already implemented	No structural change required
Player hopping	Game.hop() / Hopper	Already implemented	Not one of the four tasks

Assignment requirement	Existing location	Current status	What eventually needs changing
Coily chase	<code>move_enemies()</code>	Already implemented	Not one of the four tasks
Lives/game over	<code>lose_life()</code>	Already implemented	Not one of the four tasks
Level completion	<code>update()</code>	Already implemented	Not one of the four tasks

12. What is already prepared for us

The repository is deliberately structured so that most of the plumbing is already there.

The three empty functions are:

</> Plain text

```
cube_palette(level)
    ↓
called by draw()

on_cube_completed(cell)
    ↓
called by paint()

bonus_life_threshold()
    ↓
called by update()
```

And the one deliberate bug is:

</> Plain text

```
cube_center()
    ↓
row // 2
```

This is exactly consistent with the README's description of **one deliberate bug + three empty feature functions**.  GitHub

The official lab also says the features should be added **one by one using prompts**, with the work performed through an iterative LLM interaction and critical code review.  Lab_4_VibeCoding_handout

Bottom line

At this point, the repository requires **four targeted changes**, not a rewrite:

Plain text

1. `cube_center()`
Fix projection arithmetic
2. `cube_palette(level)`
Add level-dependent cube colors
3. `on_cube_completed(cell)`
Add completion behavior
4. `bonus_life_threshold()`
Select/return bonus-life score threshold

Everything around those functions is already wired up to use them.

No code has been modified. The next step can therefore be the actual Vibe Coding process, where we tackle the four tasks one-by-one through prompts, preserving the iterative chat history required for your lab submission.

Now implement Task 1 only: fix the deliberate pyramid projection bug.

The README indicates that the issue is in `cube_center()`, specifically the use of `row // 2` in the horizontal offset.

Before changing the code:

1. Explain briefly why `row // 2` causes the pyramid to become asymmetric.
2. Identify the minimal correction required.
3. Do not modify any other functionality.

Then make only the required change in the repository.

After modifying the code:

1. Show me exactly what was changed.
2. Explain why the correction fixes the pyramid symmetry.
3. Do not implement any of the three feature functions yet.

Worked for 7s

The required change is very small, and the repository confirms the bug is exactly where the README identifies it: `cube_center()` uses `row // 2` in the horizontal offset.  [GitHub](#)

Before the change

The current expression is effectively:

</> Python

```
col = row // 2
```

`//` performs integer division, so the fractional half-row offsets are discarded.

For example:

</> Plain text

```
row = 0 → 0
row = 1 → 0   ← should introduce a half-row offset
row = 2 → 1
row = 3 → 1   ← should introduce another half-row offset
```

That makes alternate rows receive the same horizontal correction, causing the cubes to drift sideways instead of forming a centered, symmetric pyramid. This is precisely the issue described in Task 1 of the README.  [GitHub](#)

Minimal correction

Change only:

</> Python

```
row // 2
```

to:

```
</> Python
```

```
row / 2
```

So the horizontal calculation becomes:

```
</> Python
```

```
WIDTH / 2 + (col - row / 2) * CUBE_W
```

This preserves the half-cube horizontal offset required for the isometric pyramid.

Repository modification status

I cannot safely push this change to `SETAPESU26/35_qbert` from the currently connected tools, because the GitHub repository is public for reading but there is no confirmed GitHub write connection available in this chat.

I have therefore **not falsely claimed to modify the repository**.

Please connect the GitHub integration that was surfaced for this task. Once it is connected, I can make **exactly this one-line change** to `game.py` and nothing else.

I will **not touch**:

- `cube_palette()`
- `on_cube_completed()`
- `bonus_life_threshold()`
- painting logic
- enemy logic
- scoring
- lives
- level logic

The repository's README explicitly separates Task 1 from Tasks 2–4, so we will keep the modification isolated to `cube_center()` .  GitHub

You do not need to push anything to GitHub at this stage.

The repository has already been cloned/forked locally and the game has already been run and recorded before making changes.

Proceed with Task 1 only.

Implement the deliberate bug fix identified in the README:

In game.py, inside cube_center(), change:

```
row // 2
```

to:

```
row / 2
```

Requirements:

1. Make only this change.
2. Do not modify cube_palette().
3. Do not modify on_cube_completed().
4. Do not modify bonus_life_threshold().
5. Do not modify any other gameplay logic.
6. Show me the exact change after making it.
7. Do not implement any other feature yet.

Worked for 11s

Understood. I'll keep Task 1 completely isolated.

I checked the available runtime, but the **locally cloned Q*bert repository is not accessible in this chat session**. The only uploaded repository-related file is the README, and the runtime cannot reach GitHub to obtain the cloned game.py .

So I cannot honestly claim that I modified your local game.py .

The **only change to make** is:

</> Diff

```
def cube_center(row, col):  
    return pygame.Vector2(  
-        WIDTH / 2 + (col - row // 2) * CUBE_W,
```

```
+      WIDTH / 2 + (col - row / 2) * CUBE_W,
      90 + row * CUBE_H
    )
```

Nothing else should change.

Specifically, these remain untouched:

</> Plain text

```
cube_palette()
on_cube_completed()
bonus_life_threshold()
all other gameplay logic
```

This is the exact Task 1 correction specified by the Q*bert README. [🔗 README](#)

If you **upload the current `game.py` from your cloned repository here**, I can make that one-line modification directly to the file and return the modified file, without touching anything else.

Now critically review the Task 1 implementation.

Verify:

1. The deliberate pyramid projection bug is fixed.
2. `cube_center()` uses `row / 2`.
3. No unrelated code was modified.
4. The change should produce a symmetric pyramid.

Do not make any additional changes.

Give me the verification result.
 import random
 import pygame

```
WIDTH, HEIGHT, CUBE_W, CUBE_H = 720, 560, 72, 42
SIDE = 21
ROWS, TARGET = 7, 2
HOP_TIME, HOP_HEIGHT = 0.28, 26
CELLS = {(r, c) for r in range(ROWS) for c in range(r + 1)}
DEFAULT_PALETTE = [(90, 160, 220), (190, 120, 70), (100, 210, 140)]
KEY_HOPS = {pygame.K_LEFT: (-1, -1), pygame.K_UP: (-1, 0),
```

```
pygame.K_DOWN: (1, 0), pygame.K_RIGHT: (1, 1)}
```

```
def cube_palette(level):  
    """Return a list of TARGET + 1 (r, g, b) colours for the cube stages,  
    or None for the default."""  
    palettes = [  
        DEFAULT_PALETTE,  
        [(80, 120, 220), (220, 150, 70), (120, 220, 160)],  
        [(150, 90, 220), (220, 130, 80), (100, 220, 200)],  
        [(70, 180, 220), (220, 100, 150), (180, 220, 90)],  
    ]  
  
    return palettes[(level - 1) % len(palettes)]
```

```
def on_cube_completed(cell):  
    """Called when a cube first reaches its target colour."""  
    return 25
```

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life."""  
    return 1000
```

```
def cube_center(row, col):  
    return pygame.Vector2(  
        WIDTH / 2 + (col - row / 2) * CUBE_W,  
        90 + row * CUBE_H  
    )
```

```
def neighbors(row, col):  
    return [(row - 1, col - 1), (row - 1, col), (row + 1, col), (row + 1, col  
+ 1)]
```

```
def shade(color, factor):
    return tuple(int(v * factor) for v in color)

class Hopper:
    def __init__(self, cell):
        self.cell, self.target, self.t, self.fall = cell, None, 0.0, None

    @property
    def busy(self):
        return self.target is not None or self.fall is not None

    def start(self, target):
        self.target, self.t = target, 0.0

    def update(self, dt):
        """Advance the hop; return the landing cell when a hop finishes
        on the pyramid."""
        if self.fall is not None:
            self.fall += 420 * dt
            return None
        if self.target is None:
            return None
        self.t += dt / HOP_TIME
        if self.t < 1:
            return None
        self.cell, self.target = self.target, None
        if self.cell not in CELLS:
            self.fall = 0.0
            return None
        return self.cell

    def pos(self):
        if self.target is None:
            base = cube_center(*self.cell)
            if self.fall is not None:
                base.y += self.fall
```

```
    return base
    start, end = cube_center(*self.cell), cube_center(*self.target)
    point = start.lerp(end, self.t)
    point.y -= HOP_HEIGHT * 4 * self.t * (1 - self.t)
    return point
```

```
class Game:
```

```
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.level, self.score, self.lives = 1, 0, 3
        self.reset()
```

```
    def reset(self, full=True):
        if full:
            self.level, self.score, self.lives = 1, 0, 3
            self.bonus_awarded = 0
        self.stages = {cell: 0 for cell in CELLS}
        self.state = "play"
        self.respawn()
```

```
    def respawn(self):
        self.player = Hopper((0, 0))
        self.coily, self.balls = None, []
        self.coily_timer, self.ball_timer, self.enemy_hop = 3.0, 2.0, 0.0
```

```
    def paint(self, cell):
        if cell not in self.stages or self.stages[cell] >= TARGET:
            return
        self.stages[cell] += 1
        self.score += 25
        if self.stages[cell] == TARGET:
            self.score += on_cube_completed(cell)
```

```
    def hop(self, key):
        if self.state != "play" or self.player.busy:
            return
```

```
delta = KEY_HOPS[key]
candidate = (self.player.cell[0] + delta[0], self.player.cell[1] +
delta[1])
self.player.start(candidate)

def move_enemies(self):
    if self.coily and not self.coily.busy:
        options = [n for n in neighbors(*self.coily.cell) if n in CELLS]
        goal = cube_center(*self.player.cell)
        self.coily.start(min(options, key=lambda n:
cube_center(*n).distance_squared_to(goal)))
        for ball in self.balls:
            if not ball.busy:
                ball.start((ball.cell[0] + 1, ball.cell[1] + random.randint(0,
1)))

def lose_life(self):
    self.lives -= 1
    if self.lives <= 0:
        self.state = "lose"
    else:
        self.respawn()

def update(self, dt):
    if self.state != "play":
        return
    landed = self.player.update(dt)
    if landed:
        self.paint(landed)
    threshold = bonus_life_threshold()
    if threshold and self.score // threshold > self.bonus_awarded:
        self.bonus_awarded = self.score // threshold
        self.lives += 1
    if self.player.fall is not None and self.player.fall > 260:
        self.lose_life()
    return
    self.coily_timer -= dt
```

```

if self.coily is None and self.coily_timer <= 0:
    self.coily = Hopper((0, 0))
self.ball_timer -= dt
if self.ball_timer <= 0:
    self.balls.append(Hopper((0, 0)))
    self.ball_timer = 5.0
self.enemy_hop -= dt
if self.enemy_hop <= 0:
    self.enemy_hop = 0.35
    self.move_enemies()
enemies = ([self.coily] if self.coily else []) + self.balls
for enemy in enemies:
    enemy.update(dt)
self.balls = [b for b in self.balls if b.fall is None or b.fall < 260]
if self.player.target is None and self.player.fall is None:
    for enemy in enemies:
        if enemy.fall is None and
enemy.pos().distance_to(self.player.pos()) < 26:
            self.lose_life()
            return
if all(stage >= TARGET for stage in self.stages.values()):
    self.score += 500
    self.state = "win"

def draw_cube(self, screen, cell, colors):
    cx, cy = cube_center(*cell)
    top = [(cx, cy - CUBE_H / 2), (cx + CUBE_W / 2, cy), (cx, cy +
CUBE_H / 2), (cx - CUBE_W / 2, cy)]
    color = colors[self.stages[cell]]
    left = [top[3], top[2], (cx, cy + CUBE_H / 2 + SIDE), (cx - CUBE_W
/ 2, cy + SIDE)]
    right = [top[1], top[2], (cx, cy + CUBE_H / 2 + SIDE), (cx +
CUBE_W / 2, cy + SIDE)]
    pygame.draw.polygon(screen, shade(DEFAULT_PALETTE[0],
0.45), left)
    pygame.draw.polygon(screen, shade(DEFAULT_PALETTE[0], 0.3),
right)

```

```
pygame.draw.polygon(screen, color, top)
pygame.draw.polygon(screen, (240, 240, 240), top, 1)
```

```
def draw(self, screen):
    screen.fill((18, 20, 38))
    colors = cube_palette(self.level) or DEFAULT_PALETTE
    for cell in sorted(CELLS):
        self.draw_cube(screen, cell, colors)
    for enemy in self.balls:
        pygame.draw.circle(screen, (230, 60, 60), enemy.pos() - (0,
12), 9)
    if self.coily:
        pos = self.coily.pos() - (0, 14)
        pygame.draw.circle(screen, (170, 80, 220), pos, 11)
        pygame.draw.circle(screen, (255, 255, 255), pos + (0, -2), 3)
    pos = self.player.pos() - (0, 14)
    pygame.draw.circle(screen, (250, 150, 40), pos, 13)
    pygame.draw.circle(screen, (255, 255, 255), pos + (-5, -3), 3)
    pygame.draw.circle(screen, (255, 255, 255), pos + (5, -3), 3)
    pygame.draw.circle(screen, (250, 90, 40), pos + (0, 6), 5)
    hud = self.font.render(f"Score {self.score} Lives {self.lives} Level
{self.level} R = reset", True, (240, 240, 240))
    screen.blit(hud, (10, 8))
    if self.state != "play":
        text = "LEVEL CLEAR! Press Space" if self.state == "win" else
"GAME OVER - Press R"
        label = self.font.render(text, True, (255, 255, 120))
        screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT -
40)))
```

```
def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Q*bert")
    clock = pygame.time.Clock()
    game = Game()
```

```

running = True
while running:
    dt = min(clock.tick(60) / 1000, 0.05)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key in KEY_HOPS:
                game.hop(event.key)
            elif event.key == pygame.K_r:
                game.reset()
            elif event.key == pygame.K_SPACE and game.state ==
"win":
                game.level += 1
                game.reset(full=False)
    game.update(dt)
    game.draw(screen)
    pygame.display.flip()
pygame.quit()

if __name__ == "__main__":
    main()

```

Task 1 verification result: PASS

I reviewed the code you provided without making any changes.

1. Deliberate projection bug fixed:

The `cube_center()` function now contains:

 Python

```

def cube_center(row, col):
    return pygame.Vector2(
        WIDTH / 2 + (col - row / 2) * CUBE_W,
        90 + row * CUBE_H
    )

```

The problematic integer division from the original implementation is gone.

2. `cube_center()` uses `row / 2`:

The exact relevant expression is:

 Python

```
col - row / 2
```

This is **true division**, so odd rows retain their required half-cube horizontal offset.

For example:

 Plain text

```
row 0 → 0.0  
row 1 → 0.5  
row 2 → 1.0  
row 3 → 1.5  
row 4 → 2.0
```

That produces the continuous half-row displacement required by the isometric layout.

3. No unrelated code modified: , with one important observation

For Task 1, the only relevant change visible in the code is:

 Diff

```
- (col - row // 2) * CUBE_W  
+ (col - row / 2) * CUBE_W
```

However, the code you supplied **already contains implementations for Tasks 2, 3, and 4:**

 Python

```
cube_palette(level)
```

 Python

```
on_cube_completed(cell)
```

 Python

```
bonus_life_threshold()
```

Those implementations are **not part of the Task 1 change** and I am not modifying them.

So, based strictly on the code you supplied, I can verify the Task 1 correction, but I cannot establish from this snippet whether those other implementations were present before your Task 1 change or were added separately.

4. Symmetric pyramid:

The corrected formula centers each row using a half-row offset:

 Python

```
WIDTH / 2 + (col - row / 2) * CUBE_W
```

For a row containing `row + 1` cubes, the horizontal coordinates are centered around `WIDTH / 2`.

For example, with `CUBE_W = 72`:

 Plain text

```
Row 0:           360
Row 1:         324   396
Row 2:       288   360   432
Row 3:     252   324   396   468
```

Each row is therefore centered on the same vertical axis, producing the intended triangular symmetry.

Final verification

Check	Result
Deliberate projection bug fixed	 PASS
<code>cube_center()</code> uses <code>row / 2</code>	 PASS
Task 1 correction is minimal	 PASS
Correct symmetry mathematically	 PASS

Check

Result

Other Task 2–4 functions untouched by my review

✓ No changes made

Task 1 is correctly implemented. I made no additional changes.

Now proceed to Task 2 only: implement the README-required `cube_palette(level)` feature.

Requirements:

1. Read the README requirements for `cube_palette(level)` before implementing it.
2. Implement `cube_palette(level)` so that it returns a valid list of `TARGET + 1` RGB colors, or `None` if that is the intended behavior according to the README.
3. Use different palettes for different levels, as suggested by the README.
4. Keep the implementation simple and consistent with the existing game architecture.
5. Do not modify `cube_center()`.
6. Do not implement `on_cube_completed()`.
7. Do not implement `bonus_life_threshold()`.
8. Do not modify unrelated gameplay logic.
9. After making the change, show me the exact code/diff for Task 2.
10. Explain briefly how the existing `draw()` function uses `cube_palette(level)`.
11. Do not push to GitHub yet.

Task 2 review: `cube_palette(level)`

I checked the README requirement first. It specifies that `cube_palette(level)` should:

- receive the current level,
- return a list of `TARGET + 1` RGB colors, currently 3,
- provide colors for the three cube stages,
- or return `None` to keep `DEFAULT_PALETTE`,
- with the suggested approach of using a different palette for each level.  README

Current implementation

The code you provided **already implements Task 2:**

</> Python

```
def cube_palette(level):
    """Return a list of TARGET + 1 (r, g, b) colours for the cube stages, or None for the default."""
    palettes = [
        DEFAULT_PALETTE,
        [(80, 120, 220), (220, 150, 70), (120, 220, 160)],
        [(150, 90, 220), (220, 130, 80), (100, 220, 200)],
        [(70, 180, 220), (220, 100, 150), (180, 220, 90)],
    ]

    return palettes[(level - 1) % len(palettes)]
```

Verification

This satisfies the README:

- **3 colors returned:** $TARGET + 1 = 3$ ✓
- Each entry is an RGB (r, g, b) tuple ✓
- Different levels produce different palettes ✓
- Levels beyond the four defined palettes cycle safely using % ✓
- It uses the existing DEFAULT_PALETTE for level 1 ✓
- It is simple and consistent with the existing architecture ✓
- It does not need to return None, because returning a valid custom palette is explicitly permitted by the README. ✓

How draw() uses it

The existing draw() contains:

</> Python

```
colors = cube_palette(self.level) or DEFAULT_PALETTE
```

Then every cube is drawn using:

</> Python

```
self.draw_cube(screen, cell, colors)
```

And draw_cube() selects the color corresponding to that cube's current stage:

</> Python

```
color = colors[self.stages[cell]]
```

So the flow is:

</> Plain text

```
self.level
  ↓
cube_palette(self.level)
  ↓
3-color palette
  ↓
draw()
  ↓
draw_cube()
  ↓
colors[self.stages[cell]]
```

This means the feature is already properly connected to the rendering system. No additional changes are necessary.

Exact Task 2 diff

There is no Task 2 diff to make from the code you supplied. The required implementation is already present:

</> Diff

```
def cube_palette(level):
    """Return a list of TARGET + 1 (r, g, b) colours for the cube stages, or None for the default."""
    - pass
    + palettes = [
    +     DEFAULT_PALETTE,
    +     [(80, 120, 220), (220, 150, 70), (120, 220, 160)],
    +     [(150, 90, 220), (220, 130, 80), (100, 220, 200)],
    +     [(70, 180, 220), (220, 100, 150), (180, 220, 90)],
    + ]
    +
    + return palettes[(level - 1) % len(palettes)]
```

That is the implementation currently present in the code you supplied.

Therefore I am making no additional change for Task 2, which also means `cube_center()` , `on_cube_completed()` , `bonus_life_threshold()` , and all other gameplay logic remain untouched.

Now proceed to Task 3 only: implement the README-required `on_cube_completed(cell)` feature.

Requirements:

1. Read the README requirement for `on_cube_completed(cell)` before implementing it.
2. Implement the completion behavior when a cube reaches TARGET for the first time.
3. The callback must only apply when the cube reaches its target stage, not on every hop.
4. Keep the implementation simple and consistent with the existing architecture.
5. You may use the README's suggested small completion bonus if appropriate.
6. Do not modify `cube_center()`.
7. Do not modify `cube_palette()`.
8. Do not implement `bonus_life_threshold()` yet.
9. Do not modify unrelated gameplay logic.
10. Do not push to GitHub yet.
11. After making the change, show the exact code/diff and explain how the completion callback is triggered.

Task 3: `on_cube_completed(cell)` review

I checked the README requirement first. It says `on_cube_completed(cell)` is called **when a specific cube first reaches its target stage**, not on every hop. The README suggests either a brief completion flash or a small bonus beyond the existing 25-point paint-step award.  [README](#)

Current implementation

In the code you provided, Task 3 is **already implemented**:

 Python

```
def on_cube_completed(cell):  
    """Called when a cube first reaches its target colour."""  
    return 25
```

This chooses the README's suggested **small completion bonus** approach.

How the callback is triggered

The existing `Game.paint()` logic is:

</> Python

```
def paint(self, cell):
    if cell not in self.stages or self.stages[cell] >= TARGET:
        return
    self.stages[cell] += 1
    self.score += 25
    if self.stages[cell] == TARGET:
        self.score += on_cube_completed(cell)
```

The important sequence is:

</> Plain text

```
Cube landed on
  ↓
stage += 1
  ↓
stage == TARGET ?
  ↓
YES → on_cube_completed(cell)
  ↓
+25 completion bonus
```

Because of this guard:

</> Python

```
if cell not in self.stages or self.stages[cell] >= TARGET:
    return
```

a cube that is already at `TARGET` cannot be painted again.

And because the callback is inside:

</> Python

```
if self.stages[cell] == TARGET:
```

it is triggered specifically on the hop that takes the cube **to** the target stage.

For TARGET = 2 :

</> Plain text

```
stage 0 → stage 1      +25
stage 1 → stage 2      +25 + completion bonus
stage 2 → stage 2      no callback
```

So the callback behavior satisfies the requested condition.

Exact Task 3 diff

There is **no change required** to the code you supplied because the implementation is already present:

</> Diff

```
def on_cube_completed(cell):
    """Called when a cube first reaches its target colour."""
-   pass
+   return 25
```

And the existing caller already uses the returned bonus:

</> Diff

```
self.stages[cell] += 1
self.score += 25
if self.stages[cell] == TARGET:
+   self.score += on_cube_completed(cell)
```

One repository/code discrepancy worth noting

The README says:

"Its return value is ignored."  README

But the actual `game.py` you supplied **does use the return value**:

</> Python

```
self.score += on_cube_completed(cell)
```

Therefore, for the actual code architecture, `return 25` correctly implements the README's suggested **small completion bonus**.

I have **not changed anything** in this step. `cube_center()` and `cube_palette()` remain untouched, and `bonus_life_threshold()` remains as it currently is.

Now proceed to Task 4 only: implement the README-required `bonus_life_threshold()` feature.

Requirements:

1. Read the README requirement for `bonus_life_threshold()` before implementing it.
2. Implement the function using the existing bonus-life bookkeeping already present in `Game.update()`.
3. Use the README's suggested threshold value if it is appropriate.
4. The function should return an integer score threshold, or `None` if bonus lives are disabled.
5. Do not redesign the bonus-life system because the bookkeeping is already implemented.
6. Do not modify `cube_center()`.
7. Do not modify `cube_palette()`.
8. Do not modify `on_cube_completed()`.
9. Do not modify unrelated gameplay logic.
10. Do not push to GitHub yet.
11. After making the change, show the exact code/diff and explain how the existing `update()` logic uses the returned threshold.

Task 4: `bonus_life_threshold()`

The README requires this function to:

- Take no arguments.
- Return an integer score threshold, or `None` to disable bonus lives.
- Work with the **existing bookkeeping in `Game.update()`**.
- Award a bonus life when the score crosses a multiple of that threshold.
- It specifically suggests **1000** as the threshold.  [README](#)

The existing code already has the required bookkeeping, so no redesign is necessary.

Exact Task 4 change

The function should be:

</> Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life."""  
    return 1000
```

So the exact diff from the original placeholder is:

</> Diff

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life."""  
-    pass  
+    return 1000
```

How `Game.update()` uses it

The existing code already contains:

</> Python

```
threshold = bonus_life_threshold()  
  
if threshold and self.score // threshold > self.bonus_awarded:  
    self.bonus_awarded = self.score // threshold  
    self.lives += 1
```

With:

</> Python

```
threshold = 1000
```

the behavior becomes:

</> Plain text

```
Score < 1000  
↓
```

```
No bonus life

Score reaches 1000
  ↓
1000 // 1000 = 1
  ↓
1 > bonus_awarded (0)
  ↓
bonus_awarded = 1
lives += 1
```

At 2000:

```
</> Plain text

2000 // 1000 = 2
  ↓
bonus_awarded = 2
  ↓
another life
```

The existing `bonus_awarded` bookkeeping prevents the same threshold from repeatedly awarding lives on subsequent frames.

Task 4 verification

Requirement	Result
README requirement followed	✓
Uses existing bookkeeping	✓
Threshold = 1000	✓
Returns integer	✓
Bonus-life system redesigned	✗ No
<code>cube_center()</code> changed	✗ No
<code>cube_palette()</code> changed	✗ No

Requirement	Result
on_cube_completed() changed	✗ No
Other gameplay logic changed	✗ No
GitHub push	✗ None

Task 4 implementation is therefore just the single-line `return 1000` change.

`</>` **game.py**
Code

All four README tasks have now been implemented.

Perform a final code review of the completed Q*bert implementation.

Verify all of the following:

1. Task 1:

- `cube_center()` uses `row / 2` instead of `row // 2`.
- The pyramid projection is symmetric.

2. Task 2:

- `cube_palette(level)` returns `TARGET + 1` RGB colors.
- Different levels can use different palettes.
- The existing `draw()` integration remains intact.

3. Task 3:

- `on_cube_completed(cell)` is triggered only when a cube reaches `TARGET`.
- The completion bonus is actually applied to the score.
- It is not repeatedly awarded for already completed cubes.

4. Task 4:

- `bonus_life_threshold()` returns 1000.
- The existing `Game.update()` bookkeeping awards bonus lives at

the appropriate thresholds.

- The same threshold does not repeatedly award lives.

5. Existing functionality:

- Player hopping still works.
- Falling off the pyramid still costs a life.
- Coily still moves toward the player.
- Completing all cubes still produces level completion.
- No unrelated gameplay logic was unnecessarily changed.

Do not modify any code during this review.

Give me a final verification report identifying any problems that still need to be fixed before I run the final game and record the AFTER video.

Final Q*bert Code Review

I reviewed the uploaded `game.py` as the completed implementation. I also checked that the file compiles successfully and mathematically checked the horizontal center of every pyramid row. No code was modified during this review. `</> game`

1. Task 1: Pyramid projection

Status:  PASS

`cube_center()` now uses:

`</> Python`

```
WIDTH / 2 + (col - row / 2) * CUBE_W
```

specifically `row / 2`, not `row // 2`. `</> game`

I checked the resulting row centers:

`</> Plain text`

```
Row 0: 360
Row 1: 324, 396
Row 2: 288, 360, 432
```

```

Row 3: 252, 324, 396, 468
Row 4: 216, 288, 360, 432, 504
Row 5: 180, 252, 324, 396, 468, 540
Row 6: 144, 216, 288, 360, 432, 504, 576

```

Every row has a center of **360**, which is $\text{WIDTH} / 2$. Therefore the pyramid is horizontally symmetric.

Task 1: PASS.

2. Task 2: `cube_palette(level)`

Status:  **PASS**

TARGET = 2 , so the function needs to provide:

 **Plain text**

```
TARGET + 1 = 3 colors
```

The implementation provides four palettes, each containing exactly three RGB tuples.  game

For example:

 **Python**

```

[
    (80, 120, 220),
    (220, 150, 70),
    (120, 220, 160)
]

```

Different levels can select different palettes through:

 **Python**

```
(level - 1) % len(palettes)
```

The existing rendering integration is intact:

 **Python**

```
colors = cube_palette(self.level) or DEFAULT_PALETTE
```

and then:

`</>` Python

```
self.draw_cube(screen, cell, colors)
```

So the feature is actually connected to rendering. `</>` game

Task 2: PASS.

3. Task 3: `on_cube_completed(cell)`

Status:  PASS

The callback returns:

`</>` Python

```
return 25
```

`</>` game

More importantly, `paint()` only calls it after incrementing the cube's stage and checking:

`</>` Python

```
if self.stages[cell] == TARGET:
    self.score += on_cube_completed(cell)
```

`</>` game

The earlier guard:

`</>` Python

```
if cell not in self.stages or self.stages[cell] >= TARGET:
    return
```

prevents already-completed cubes from being painted again.

Therefore:

</> Plain text

```
stage 0 → stage 1      +25
stage 1 → stage 2      +25 + completion bonus
stage 2 → stage 2      no additional award
```

The completion bonus is actually added to the score.

Task 3: PASS.

4. Task 4: Bonus life

Status:  PASS

The function returns exactly:

</> Python

```
return 1000
```

</> game

The existing `Game.update()` bookkeeping remains:

</> Python

```
threshold = bonus_life_threshold()

if threshold and self.score // threshold > self.bonus_awarded:
    self.bonus_awarded = self.score // threshold
    self.lives += 1
```

</> game

This means the thresholds work as intended:

</> Plain text

```
1000 → first bonus life
2000 → second bonus life
3000 → third bonus life
...
```

`bonus_awarded` records how many thresholds have already been awarded, so the same threshold isn't repeatedly rewarded every frame.

Task 4: PASS.

5. Existing gameplay functionality

Player hopping:  PASS

`Game.hop()` still calculates the movement candidate from `KEY_HOPS` and starts the player's `Hopper`.

`</>` game

Falling off the pyramid:  PASS

`Hopper.update()` still checks:

`</>` Python

```
if self.cell not in CELLS:
    self.fall = 0.0
```

and `Game.update()` subsequently detects sufficient fall distance and calls:

`</>` Python

```
self.lose_life()
```

`</>` game `</>` game

Coily movement:  PASS

Coily still calculates the player's current screen position:

`</>` Python

```
goal = cube_center(*self.player.cell)
```

and selects the available neighboring cell minimizing its distance to that goal. 🔗 game

Thus the Task 1 `cube_center()` correction also consistently affects Coily's spatial calculations rather than breaking them.

Level completion: ✅ **PASS**

The existing logic remains:

🔗 Python

```
if all(stage >= TARGET for stage in self.stages.values()):
    self.score += 500
    self.state = "win"
```

🔗 game

Space then advances the level:

🔗 Python

```
game.level += 1
game.reset(full=False)
```

🔗 game

Code validity: ✅ **PASS**

I also checked the complete uploaded file for Python syntax errors. It **compiles successfully**.



Final report

Requirement	Result
Task 1: row / 2	✅ PASS
Task 1: symmetric pyramid	✅ PASS
Task 2: 3 RGB colors	✅ PASS
Task 2: different level palettes	✅ PASS

Requirement	Result
Task 2: draw() integration	✓ PASS
Task 3: triggered at TARGET	✓ PASS
Task 3: bonus applied	✓ PASS
Task 3: no repeated completion bonus	✓ PASS
Task 4: threshold = 1000	✓ PASS
Task 4: existing bookkeeping	✓ PASS
Task 4: no repeated threshold awards	✓ PASS
Player hopping	✓ PASS
Falling costs a life	✓ PASS
Coily moves toward player	✓ PASS
Level completion	✓ PASS
Python syntax	✓ PASS

Final verdict

I don't see a code-level problem that needs to be fixed before the final run.

The four README tasks are implemented, the existing gameplay paths remain intact, and the uploaded `game.py` compiles successfully.

You can proceed to **run the completed game and record the 10-second AFTER video**. The README specifically requires the AFTER recording to show the bug fixed and the new features working.

 README

One practical point for the recording: make sure the AFTER video visibly demonstrates enough of the new behavior to establish the work, particularly the **symmetric pyramid and level-dependent cube colors**, and, if feasible within 10 seconds, the completion/bonus-life behavior.